Holly Pasquinelli
Lab 2

Understanding and Analysis of Image Files

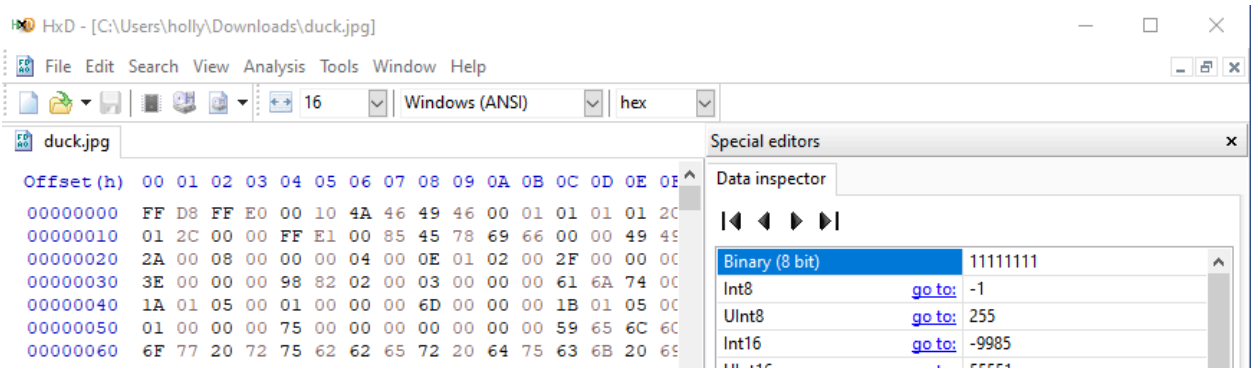
Non-technical Summary

In this lab we explored file analysis using HxD, and hashing using different environments. In digital forensics, it is important to understand all elements of files including all data in the file. For example, a file may appear to be a PDF or word document based on its icon, however, the file may be disguised as an executable. We can analyze the signature of the file to verify file type.

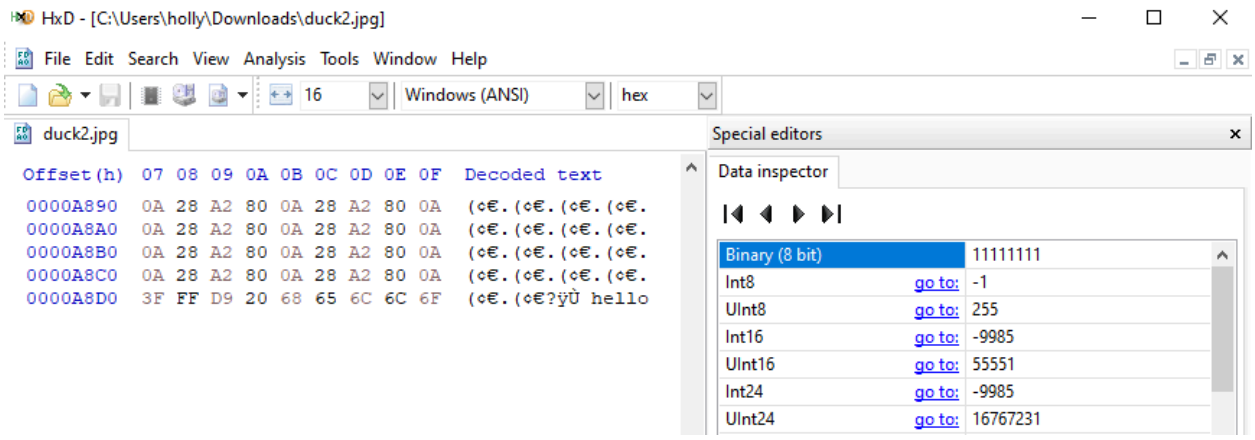
In the event of a criminal investigation, it is important that files are hashed to ensure the data has not been modified, or tampered with. Hashing uses an algorithm to convert data into a hash value, as a hexadecimal. The most important part of a hash is that it is irreversible. This means that we cannot get back to the original input from its hash value. In order to be used as evidence in court, the moment a file is discovered, it is required that the file and all of its data is secured. This ensures that even through copies and transfers, the duplicates of the file are exact copies of the original. This can be verified by comparing the hash values between the original and copy. Without a hash, a file cannot be considered for evidence, because there will not be a way to prove the original has not been modified since discovery.

Technical Report

We started the lab by using HxD, where we analyzed an image. Once we uploaded an image, we were able to see the file signature in the first line. As we can see, the image I used called duck.jpg is in fact a jpeg image. The signature for a jpeg image is FF D8 FF E0 00 10 4A 46 49 46 00 01. We can see this signature matches the first line in HxD. Here, we verified the file type with the matching signature for jpeg. We can do this for all kinds of different files by looking up their signature, and matching it to the first line in HxD when we upload the file.



After analyzing the signature, we notice the data for each pixel. If we scroll to the end of this data, and type a secret message, this message will not be displayed in the picture. This is because the message is entered after all the data that will be displayed in the pixels. This is important because the image may not appear to be modified, but there is information hidden within the data of the image that could potentially be important to forensic analysts in the event of a crime.



In the next parts of the lab, we focused on hashing. We learned there are many methods of obtaining a hash value. We started with hashing using the command prompt in windows. This is done by using the command line utility certutil. SHA1 is the algorithm used to compute the hash values we see after the command is executed. Once we compute the hashes of both the original and modified files, we see that the hash values are different. We can expand on the certutil command by using FORFILES. FORFILES and its associated switches allow us to compute hash values for a set of files all at once via one command. As we can see below, we use /C which runs the command certutil -hashfile on each file specified by /M. /M specifies a search mask, so here we are looking for any jpeg files by specifying .jpg in the search mask. With this, the command executed below computes hash values for all jpeg files in my downloads folder. We can see that this returns two sets of hash values, one for the original image duck.jpg, and one for the modified image duck2.jpg. Again, we see these hash values are different. This shows us that the data in duck2.jpg has been modified, and therefore is not the same as the original image.

```
C:\Users\holly\Downloads>FORFILES /M *.jpg /C "cmd /c certutil -hashfile @FILE"

SHA1 hash of duck.jpg:
c1e978def3c9d21554533a885560efef8bda4c68
CertUtil: -hashfile command completed successfully.
SHA1 hash of duck2.jpg:
c8d6a91be14d8edad4eb51607462779840b4494d
CertUtil: -hashfile command completed successfully.
```

There are other switches associated with FORFILES such as /P, /S, /D and variables @FNAME, @EXT, @PATH, @RELPATH, @FSIZE, @FDATE, @FTIME. /P changes the directory we are searching in, /S will search recursively in subdirectories, and /D specifies a last modified date using + as later than or on the given date, and - as earlier than or on the given date. Variables file names, extensions, paths, file size, and last modified date and time. These switches and variables allow us to quickly and efficiently search for and apply commands to a lot of files from a specific criteria.

```
C:\Users\holly\Downloads>FORFILES /M *.jpg /C "cmd /c certutil -hashfile @FILE" /D -365
ERROR: No files found with the specified search criteria.

C:\Users\holly\Downloads>
```

In the image above we can see that there are no images that meet the specified criteria of the previous year. In the image below, we can see that by entering +/- and today's date, that we receive the same results. This is because /D +/- specifies the last modified date inclusive of the date listed. So, if we wanted to exclude today, we would have to list the day before in order to exclude any files that were modified today.

```
C:\Users\holly\Downloads>FORFILES /M *.jpg /C "cmd /c certutil -hashfile @FILE" /D +09/19/2025

SHA1 hash of duck.jpg:
c1e978def3c9d21554533a885560efef8bda4c68
CertUtil: -hashfile command completed successfully.
SHA1 hash of duck2.jpg:
2717e22cde5c284d2cf8b6d2df96e9fe9aea8770
CertUtil: -hashfile command completed successfully.
SHA1 hash of flower.jpg:
906d946d21bd94986424cc2be4cce29971013d6d
CertUtil: -hashfile command completed successfully.

C:\Users\holly\Downloads>FORFILES /M *.jpg /C "cmd /c certutil -hashfile @FILE" /D -09/19/2025

SHA1 hash of duck.jpg:
c1e978def3c9d21554533a885560efef8bda4c68
CertUtil: -hashfile command completed successfully.
SHA1 hash of duck2.jpg:
2717e22cde5c284d2cf8b6d2df96e9fe9aea8770
CertUtil: -hashfile command completed successfully.
SHA1 hash of flower.jpg:
906d946d21bd94986424cc2be4cce29971013d6d
CertUtil: -hashfile command completed successfully.
```

We can achieve the same digest for the two image files duck.jpg and duck2.jpg by using Windows PowerShell. We can use the command GET-FILEHASH, and specify the algorithm using -Algorithm. In the lab we again used SHA1, and then used a file mask to search all jpeg images. Once executed the command returns the algorithm along with the hash value, and path of the file.

We can also use hashlib in python to compute the hash values of the two image files. We open the files, and then read the data using .read. By printing hashlib.sha1, we compute the hash values using SHA1 algorithm again. The output is simply the hash value of each image on each line.

Another package we explored using Python is PIL, or the Pillow Image Processing Package. In this case, we open the image files and convert them to bytes.

This converted object is then run through the hashlib command and then printed. We see that both hash values are the same. Although the images to the eye are the same, we know there are differences in the data of each image file. This is not reflected using the PIL method when it comes to computing hash values. This should never be used when it comes to digital forensics. Clearly, it would create a misconception that files being compared to one another are exact replicas, when in fact their data composition is different.

Using Python, we can also compute the hash values of image files with seemingly undetectable secrets hidden. In the lab, we created a secret and embedded it into an image. We can see again that there are no visible differences between the two images. In this case, we can again use the PIL package, but this time to find the difference between the two images. We convert the difference into an array, and show the indices with a difference, where the difference is not equal to zero. We are then able to print the indices where the secret message was hidden in the image data. We then printed the hash values for both images in the SHA1 and SHA256 values.

In the last part of the lab we explored fuzzy hashes. Fuzzy hashes are also called Context Triggered Piecewise Hashes (CTPH). These hashes compare files and allow us to understand the files similarities. This means that files that are extremely similar should have a similarity level of 100. Files that are not at all similar should have a similarity level of 0. We can use fuzzy hashing via the package ppdeep. To compute the hash value from a file, we use the command ppdeep.hash_from_file. We use the compare method to give us the level of similarity from the hash values. It is difficult to decipher the level of similarity value. This is because 0 and 100 could also mean extremely similar, or extremely different, respectively. With that being said, there is really no way to determine what similarity values between 0 and 100 really mean. This is not a solid way to compare data in files, or compute hash values. Unlike cryptographic hashes, we will not have unique values. Two sets of files may have the same level of similarity. We can see there is a lot of discrepancy, and the fuzzy hash is not simply black or white as we require it to be in digital forensics cases.

As we can see, there are various different ways we can compute hash values. It is important to note that for the purpose of digital forensics, we should use cryptographic hashing. We have seen these algorithms grow stronger as they have to compete with higher levels of computing. As SHA1 has been deemed weak, we have seen the rise of SHA3. Cryptographic hashing is extremely secure, and demonstrates the intended purpose of using a hash. In any case of hashing, it is irreversible, and we cannot get back to the original data by decoding the hash. Cryptographic hashing is very sensitive. This means that even the slightest change in a file's data will completely alter the hash value between two files with high similarity. Whereas non cryptographic and fuzzy hashing are not known for their high security levels, they are much faster than cryptographic hashing.

References

https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/forfiles